Here's a draft design for Lock In that fits the requirements table. Reword it in your own voice before you paste it in, since the worksheet requires that. I'm giving you the structure, not finished answers.

State and second dataclass

Item	Draft
`State` attributes (5)	`name: str` (set on the welcome page), `focus_category: str` (picked from a SelectBox), `goals: list[Goal]` (grows and changes through the site), `day_rating: int` (1 to 5, set at check-in), `notes: str` (set at check-in)
Second dataclass `Goal`	`name: str`, `category: str`, `done: bool`. Only `done` changes after creation, when the user checks a task off

Compared with your Week 10 worksheet, I made the rating one per day, not per task, and that's the simpler choice. I'd also add an optional `message: str` to `State` for error text like "Task name can't be blank."

Pages (5) and routes (7+)

Page	Route(s)
Welcome: enter name and pick a focus category	`index`, `start_day`
Task list: shows tasks, progress bar, score, status	`tasks_page`, `toggle_task`
Add task: name and category	`add_task_page`, `save_task`
Check-in: day rating and notes	`checkin_page`, `save_checkin`
Summary: final score, tasks done, notes	`summary_page`, `reset_day`

That gives you 5 pages and 9 routes, so you have some room to cut if needed.

Other requirements

Requirement	Draft
User input (3+)	TextBox for name, SelectBox for category, TextBox for task name, CheckBox (or button) for done, SelectBox for rating, TextArea for notes
If statements (3+)	Reject a blank task name; reject a duplicate task name; pick the status label (Locked in / On track / Need to push) from the score; show “No tasks yet” when the list is empty
Loop (1+)	Loop over `state.goals` to count the done tasks, and to build the task rows on the page

Score formula (decide this now)

A simple option is `percent_done = done_count / total * 100`, then `score = (percent_done + day_rating * 20) / 2`. Pick the cutoffs for the status labels yourself, such as 80 and up for Locked in, 50 and up for On track, and below that Need to push.

Site diagram tips

Draw 5 boxes, one per page, with arrows labeled by route name (like `save_task`). Put the 5 State fields in a box off to the side. Keep it quick and rough, since 15 minutes is enough.

Section 0 (planning questions)

These questions are about you, so I’d write them yourself. I can help if you tell me your answers first. A few prompts:

- Concepts: Which class concepts do you expect to use (lists, loops, dataclasses, if statements)?
- Easiest and hardest: You already said that updating the right task and the score math worry you.
- Build order: Probably State and Goal first, then add task, then the task list, then check-in and score.

Once again, paste this whole conversation into the LLM Conversation section at the bottom. Want me to help sketch the diagram layout or tighten the score formula next?

tell me why you would choose to do these thing this certain way and does it maintain the rubric

I made each choice for one of three reasons: it meets a rubric line, it keeps the build small, or it matches what you wrote in Week 10. One caveat: I've only seen the requirements table in your worksheet, not the full rubric in Canvas, so check the Canvas requirements once before you lock this in.

Why I chose these things

- Five State fields, not four. The minimum is four, and going one over gives you a cushion. If you cut notes later, you still pass.
- A `Goal` dataclass with `name`, `category`, and `done`. That's exactly the "details I need" you listed in Week 10. Leaving out due dates, reminders, and streaks keeps it simple, and it's the abstraction your Part 3 already argues for.
- One day rating, not one per task. A rating per task would force you to store a rating on every `Goal` and average them. A single `day_rating` on `State` is easier to update and easier to explain.
- Separate "page" and "save" routes. Drafter pages show things, and routes that handle a button press do the work. Splitting them gives you 9 routes, so you clear the 7-route minimum even if you merge or cut two.
- The loop over `state.goals`. The rubric requires a loop over your list of dataclass instances. Counting done tasks and building the task rows are natural reasons for it, so the loop isn't artificial.
- The if statements. Blank and duplicate checks guard the input, and the status label changes what the user sees. That makes each one a real change in logic, which is what the table asks for.

Rubric check, based on your worksheet

Requirement	Met?	Why
Purpose, 1 to 2 sentences	Yes	Daily task tracker with a day score

State with 4+ attributes, one a list of instances	Yes	5 fields, `goals` is `list[Goal]`
Second dataclass, meaningfully modified	Yes	`Goal.done` changes in `toggle_task`
7+ routes	Yes	9
5+ pages	Yes, with a caveat	5 pages if the save routes redirect to existing pages
3+ user inputs	Yes	TextBox, SelectBox, CheckBox, TextArea
3+ if statements	Yes	4
1+ loop over the list of instances	Yes	Loop over `state.goals`

You also mentioned a rubric note that 4 State fields must each be changed in a route. In this design: `name` and `focus_category` change in `start_day`, `goals` in `save_task` and `toggle_task`, and `day_rating` and `notes` in `save_checkin`. All five are covered.

Two risks to watch

1. If your instructor counts only routes that return a distinct page, you'd have 5 pages, which is exactly the minimum. I'd confirm this in Canvas, and consider a sixth page, like a simple "About" or "Settings" page, if you want a safety margin.
2. The score formula is yours to define. If it's too fuzzy in the build, the status label if statement gets hard to test.

If you want, paste the Canvas requirements here and I'll check this design line by line.

the task is still the task track lock in

Yes, the design is still the Lock In task tracker. Nothing I suggested changed the core idea, which is that you add tasks for the day, check them off, rate how the day is going, and get a day score and a status of Locked in, On track, or Need to push.

The only changes from your Week 10 worksheet are small:

- The day rating is one per day, not one per task.
- The notes and categories are optional extras you can cut if time gets tight.
- I split the work into separate page routes and save routes so you clear the route minimum.

If any part of the design feels like it drifted from what you want, tell me which part and I'll change it. Otherwise, the next step is the site diagram: five page boxes, arrows labeled with route names, and the State fields off to the side.

Final Project Design Phase Computer Science - Week 11 Oct 3, 2026 - Version 0.0.4 For complete instructions and project ideas, refer to the assignment in Canvas. Specifically, review the final project requirements before you continue below. If you have not reviewed the project requirements

pasted

The design holds together and meets the requirements. These are the problems to fix before you submit.

Fixes needed

1. Pages: you only describe 4. Home, Tasks, Checkin, and Progress are listed, but the minimum is 5. Your routes include `profile_page`, so add Profile as the fifth page, with a line like "set your name and focus category."
2. State attributes need types. The table asks for each attribute's type, and you only gave descriptions. Add them: `goals: list[Goal]`, `user_name: str`, `daily_note: str`, `day_rating: int`, `focus_category: str`.
3. Second dataclass wording. "Goal:str" should read like "Goal has `name: str`, `category: str`, `done: bool`."
4. Unfinished sentence. "Checkbox: whether the task" stops mid-thought. Finish it, for example "whether the task is done."
5. Garbled if statement. "No goal check in the task page and the check in page to show a message instead of the empty message" is hard to follow. Try "If the goals list is empty, the tasks page and check-in page show a 'No tasks yet' message instead of an empty list."
6. Loop descriptions. "Goal.line" isn't a clear name. Use the actual function names you plan to write, like `count_done` and the function that builds each task row.
7. Typos. "rate how to day is going" should be "rate how the day is going."

The LLM Conversation section is incomplete

- The pasted text includes interface lines like “Added to memory lock-in-final-project.md” and “Verifying the state fields against the requirements.” Those aren’t part of the conversation, so delete them.
- It starts partway through. The worksheet says to paste every part, so include the whole chat from the beginning: the four layout options, the study-hours question, the Week 10 feedback, and this message too. Leaving parts out risks the zero the worksheet warns about.
- Label who said what (“Me:” and “Claude:”) so it reads clearly.

Still missing

- The Site Diagram screenshot or photo. Sketch five page boxes, label the arrows with route names like `save_checkin`, and put the State fields in a box on the side. Fifteen minutes is plenty.

What’s good

- Your State fields are each changed in a route: `save_profile` sets the name and category, `add_goal` and `save_checkin` change goals, and `save_checkin` sets the rating and note.
- You have 8 routes, 5 if statements, and 3 loops, all above the minimums.
- Your Section 0 answers are specific and in your own voice.

One thing to decide before Week 12: the exact score formula and status cutoffs. Your `progress_page` and the status if statement depend on it.

Deciding which goals and check-in options to keep
You stopped Claude’s response

but it still have to pass everything from the rubric

Ran 2 commands, created a file, shared files

I made a leaner version in `main_short.py` that still meets every rubric item. It’s about 90 lines shorter than the last one, and all the unit tests pass. I called every route directly, but I haven’t launched it in a browser, so run it in Thonny before you record. Rename it to `main.py` for deployment.

What I cut to make it easier to explain

- Categories, the focus category, and the category breakdown are gone, so `Goal` is just `name` and `done`.
- The default-value helpers are gone, and the check-in page no longer needs default arguments.
- The profile page now only sets your name.

Rubric check

Requirement	Status
7+ routes	Yes, 8
5+ pages	Yes: Home, Profile, Tasks, Check-in, Progress
State with 4+ fields, list of `Goal`, second dataclass	Yes, 5 fields
4 State fields modified in routes	Yes: `goals`, `user_name`, `day_rating`, `daily_note` (`message` too)
3+ inputs	Yes: TextBox, SelectBox, CheckBox, TextArea
3+ if statements	Yes: blank task, duplicate task, empty list, greeting, status
Loop over the list	Yes: `count_done`, the tasks page loop, `goal_names`, `save_checkin`
No globals	Yes, only the constant `RATINGS`
Types, docstrings, tests per route	Yes
`set_site_information(...)`	Yes, at the top

A 5-minute video plan

1. About 30 seconds: show the live site and say what Lock In does.

2. About 60 seconds: explain the two dataclasses, `Goal` and `State`.
3. About 90 seconds: walk through the helper functions and the score formula.
4. About 90 seconds: walk through the routes in the order a user would click them: profile, add task, check-in, progress.
5. About 30 seconds: show the unit tests and mention the if statements and loops.

Before you record, update the `links` entry in `set_site_information` with your real repo URL. The repository items are still yours: the worksheets, this chat saved as a file, the video link, and descriptive commit messages.

Main short